EXPERIMENT 3: MODEL PLANNING AND BUILDING

AIM: To implement model planning and building on the Advertising dataset by exploring and preprocessing data, training Linear Regression, Polynomial Regression, and Support Vector Regression models, evaluating their predictive performance using metrics (MAE, MSE, RMSE), and segmenting the data using K-Means Clustering with visual plots.

ALGORITHM:

1. Load the Advertising dataset from Kaggle into a DataFrame.
2. Explore and preprocess the data by checking for null values and data types, selecting features (TV, Radio, Newspaper) and the target variable (Sales), and plotting a pair plot on the features.
3. Split the dataset into training and testing sets using train_test_split().
4. Train a Linear Regression model on the training data and make predictions on the test set.
5. Train a Polynomial Regression model by constructing a pipeline with PolynomialFeatures(degree=2) and LinearRegression, fitting it on the training set, and predicting on the test set.
6. Train a Support Vector Regression (SVR) model using an RBF kernel on the training set and compute predictions on the test set.
7. Evaluate the models by computing MAE, MSE, and RMSE using mean_absolute_error, mean_squared_error, and np.sqrt(), and visually compare all models on a single bar graph.
8. Apply K-Means Clustering by fitting KMeans(n_clusters=3) on the feature set and assigning the resulting cluster labels to the dataset.
9. Visualize the clusters using seaborn.scatterplot() (e.g., TV vs Sales colored by cluster label).

Step 1: Load the Advertising Dataset

```
#EXP_NO_3
#SANJAY_S_241801246
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans


df = pd.read_csv("advertising.csv")
df.head(5)
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

Step 2: Explore and Preprocess Data

```
#EXP_NO_3
#SANJAY_S_241801246
print(df.shape)
```

```
(200, 4)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
```

```
print(df.isnull().sum())
```

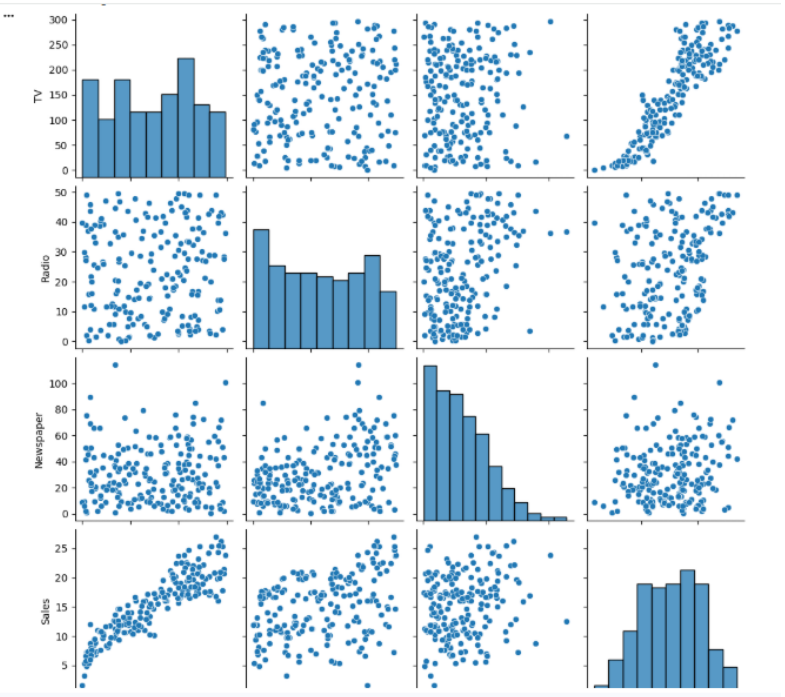
```
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64
```

```
dtype: int64
```

```
print(df.describe())
```

```
               TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   15.130500
std     85.854236   14.846809   21.778621    5.283892
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   11.000000
50%    149.750000   22.900000   25.750000   16.000000
75%    218.825000   36.525000   45.100000   19.050000
max    296.400000   49.600000  114.000000   27.000000
```

```
sns.pairplot(df)
```



Step 3: Split the Dataset

```
#EXP_NO_3
#SANJAY_S_241801246
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (160, 3)
Testing Data: (40, 3)
```

Step 4: Train Linear Regression

```
#EXP_NO_3
#SANJAY_S_241801246
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
linear_pred = linear_model.predict(X_test)
print("Linear Regression Predictions:")
print(linear_pred[:5])
```

```
Linear Regression Predictions:
[17.0347724  20.40974033 23.72398873  9.27278518 21.68271879]
```

Step 5: Train Polynomial Regression

```
#EXP_NO_3
#SANJAY_S_241801246
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
poly_model.fit(X_train, y_train)
poly_pred = poly_model.predict(X_test)
print("Polynomial Regression Predictions:")
print(poly_pred[:5])
```

```
Polynomial Regression Predictions:
[17.562438   21.96285664 22.07580481  7.34916293 23.23684435]
```

Step 6: Train Support Vector Regression (SVR)

```
#EXP_NO_3
#SANJAY_S_241801246
svr_model = SVR(kernel='rbf')
svr_model.fit(X_train, y_train)
svr_pred = svr_model.predict(X_test)
print("SVR Predictions:")
print(svr_pred[:5])
```

```
SVR Predictions:
[17.15330196 19.49875585 20.96245814  8.60209215 19.88553201]
```

Step 7: Evaluate the Models

```
#EXP_NO_3
#SANJAY_S_241801246
models = {
    "Linear Regression": linear_pred,
    "Polynomial Regression": poly_pred,
    "SVR": svr_pred
}

mae_values = []
mse_values = []
rmse_values = []

for name, pred in models.items():
    mae = mean_absolute_error(y_test, pred)
    mse = mean_squared_error(y_test, pred)
    rmse = np.sqrt(mse)

    mae_values.append(mae)
    mse_values.append(mse)
    rmse_values.append(rmse)

    print(name)
    print("MAE :", mae)
    print("MSE :", mse)
    print("RMSE:", rmse)
    print()

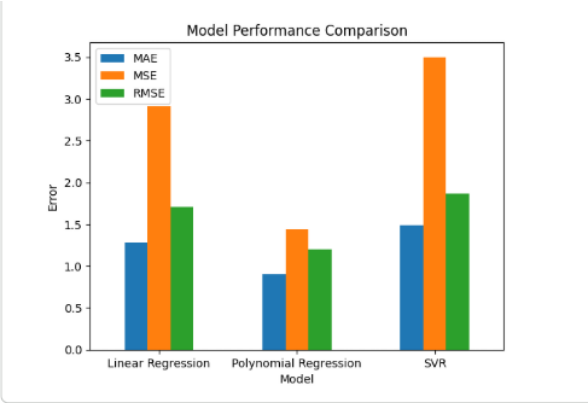
# Comparison bar graph
results = pd.DataFrame({
    "Model": list(models.keys()),
    "MAE": mae_values,
    "MSE": mse_values,
    "RMSE": rmse_values
})

results.set_index("Model").plot(kind="bar")
plt.title("Model Performance Comparison")
plt.ylabel("Error")
plt.xticks(rotation=0)
plt.show()
```

```
Linear Regression
MAE : 1.2748262109549338
MSE : 2.907756910271089 6
RMSE: 1.7052146229349223

Polynomial Regression
MAE : 0.9033713382792442
MSE : 1.44254263690159
RMSE: 1.2010589647896517

SVR
MAE : 1.4836223881587627
MSE : 3.499087197255254
RMSE: 1.8705847206836834
```



Step 8: Apply K-Means Clustering

```
#EXP_NO_3
#SANJAY_S_241801246
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(
    df[['TV', 'Radio', 'Newspaper']]
)
print(df[['TV', 'Radio', 'Newspaper', 'Cluster']].head())
```

```
      TV  Radio  Newspaper  Cluster
0  230.1   37.8       69.2        0
1   44.5   39.3       45.1        1
2   17.2   45.9       69.3        1
3  151.5   41.3       58.5        2
4  180.8   10.8       58.4        2
```

Step 9: Visualize the Clusters

```
#EXP_NO_3
#SANJAY_S_241801246
sns.scatterplot(
    x='TV',
    y='Sales',
    hue='Cluster',
    data=df
)
plt.title("K-Means Clusters: TV vs Sales")
plt.show()
```



RESULT: The model planning and building exercise was successfully implemented. Linear Regression, Polynomial Regression, and Support Vector Regression models were built and evaluated using MAE, MSE, and RMSE metrics, and data clustering was performed and visualized using K-Means.